1 Part 2: Data Visualization and Business Analysis

1.1 Story 1: Channel Efficiency Is Declining While Return Risk Stays Flat

1) Visualization Code (Python)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

sns.set_style("whitegrid")

data_dir = Path("./Data") if Path("./Data/orders.csv").exists() else
    Path("./")
orders = pd.read_csv(data_dir / "orders.csv", parse_dates=["order_date"
    ])
web = pd.read_csv(data_dir / "web_traffic.csv", parse_dates=["date"])
order_items = pd.read_csv(data_dir / "order_items.csv", low_memory=
    False)
products = pd.read_csv(data_dir / "products.csv")
returns = pd.read_csv(data_dir / "returns.csv")

line = (
    order_items
    .merge(orders[["order_id", "order_source"]], on="order_id", how="
        left")
    .merge(products[["product_id", "cogs"]], on="product_id", how="left
        ")
)
line["discount_amount"] = line["discount_amount"].fillna(0)
line["cogs"] = line["cogs"].fillna(line["unit_price"] * 0.6)
line["net_sales"] = line["quantity"] * line["unit_price"] - line["
    discount_amount"]
line["gross_profit"] = line["net_sales"] - line["quantity"] * line["
    cogs"]

orders_m = (
    orders.assign(month=orders["order_date"].dt.to_period("M").dt.
        to_timestamp())
    .groupby(["month", "order_source"], as_index=False)
    .agg(orders=("order_id", "nunique"))
)
web_m = (
    web.assign(month=web["date"].dt.to_period("M").dt.to_timestamp())
    .groupby(["month", "traffic_source"], as_index=False)
    .agg(sessions=("sessions", "sum"), bounce=("bounce_rate", "mean"))
)
channel = orders_m.merge(
    web_m,
    left_on=["month", "order_source"],
    right_on=["month", "traffic_source"],
    how="inner"
)
channel["orders_per_1k_sessions"] = 1000 * channel["orders"] / channel[
    "sessions"].replace(0, np.nan)

eff = channel.groupby("order_source", as_index=False).agg(
    conv_eff=("orders_per_1k_sessions", "mean"),
```

```
    bounce=("bounce", "mean")
)

econ = line.groupby("order_source", as_index=False).agg(
    net_sales=("net_sales", "sum"),
    gross_profit=("gross_profit", "sum"),
    orders=("order_id", "nunique"),
    sold_qty=("quantity", "sum")
)
econ["aov"] = econ["net_sales"] / econ["orders"]
econ["gp_per_order"] = econ["gross_profit"] / econ["orders"]

ret = (
    returns.merge(orders[["order_id", "order_source"]], on="order_id",
        how="left")
    .groupby("order_source", as_index=False)["return_quantity"].sum()
)

plot_df = eff.merge(econ, on="order_source", how="left").merge(ret, on=
    "order_source", how="left")
plot_df["return_quantity"] = plot_df["return_quantity"].fillna(0)
plot_df["return_rate"] = plot_df["return_quantity"] / plot_df["sold_qty
    "].replace(0, np.nan)

fig, ax = plt.subplots(figsize=(11, 6))
sns.scatterplot(
    data=plot_df,
    x="conv_eff",
    y="return_rate",
    size="net_sales",
    hue="order_source",
    sizes=(150, 1000),
    palette="tab10",
    ax=ax
)
for _, r in plot_df.iterrows():
    ax.annotate(r["order_source"], (r["conv_eff"], r["return_rate"]),
        xytext=(5, 5), textcoords="offset␣points")

ax.set_title("Channel␣Matrix:␣Conversion␣Efficiency␣vs␣Return␣Rate")
ax.set_xlabel("Orders␣per␣1,000␣sessions")
ax.set_ylabel("Return␣rate")
ax.legend(title="Order␣Source", bbox_to_anchor=(1.02, 1), loc="upper␣
    left")
plt.tight_layout()
plt.show()
```

2) Data Analysis (Descriptive, Diagnostic, Predictive, Prescriptive) **Descriptive & Diagnostic.** Social media is the most efficient channel (10.40 orders per 1,000 sessions), while organic search is lower at 7.39. Return rates are nearly flat across channels (3.33%–3.46%), so the performance gap is driven by conversion efficiency rather than post-purchase quality.

Predictive. Monthly conversion-efficiency trends are negative across all channels, implying rising CAC under the current media mix.

Prescriptive. Reallocating 100,000 sessions from the weakest to the strongest channel can generate 300.84 incremental orders, or +7.29 million in net revenue and +0.71 million in gross profit. Weekly spend should be increased only for channels whose *orders_per_1k_sessions* exceed

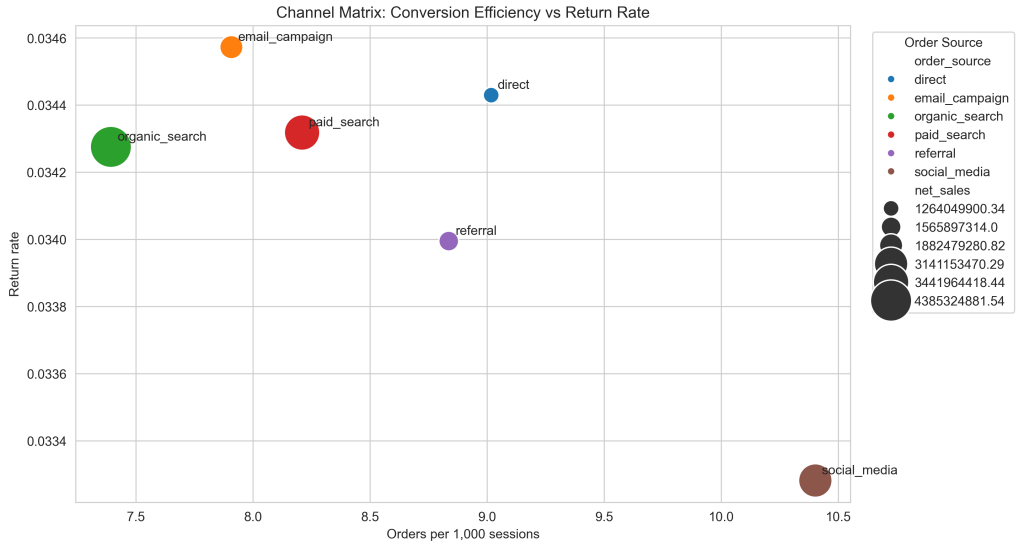

Figure 1: **Channel quality matrix.** Bubble size indicates net sales. The x-axis shows conversion efficiency (orders per 1,000 sessions) and the y-axis shows return rate. Conversion efficiency varies substantially across channels, whereas return rates remain nearly flat, suggesting that budget reallocation should prioritize conversion yield.

the rolling 4-week median and whose *return_rate* remains below the 75th percentile.

3) Figure caption (NeurIPS format)

1.2 Story 2: Discounts Above 10% Are Eroding Margin

1) Visualization Code (Python)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

sns.set_style("whitegrid")

data_dir = Path("./Data") if Path("./Data/order_items.csv").exists()
    else Path("./")
order_items = pd.read_csv(data_dir / "order_items.csv", low_memory=
    False)
products = pd.read_csv(data_dir / "products.csv")
promotions = pd.read_csv(data_dir / "promotions.csv")
inventory = pd.read_csv(data_dir / "inventory.csv", parse_dates=["
    snapshot_date"])

latest_inv = inventory[inventory["snapshot_date"] == inventory["
    snapshot_date"].max()][["product_id", "overstock_flag"]]

line = (
    order_items
    .merge(products[["product_id", "cogs", "category", "segment"]], on=
        "product_id", how="left")
    .merge(latest_inv, on="product_id", how="left")
)
```

```
line["discount_amount"] = line["discount_amount"].fillna(0)
line["cogs"] = line["cogs"].fillna(line["unit_price"] * 0.6)
line["overstock_flag"] = line["overstock_flag"].fillna(0)

line["gross_sales"] = line["quantity"] * line["unit_price"]
line["net_sales"] = line["gross_sales"] - line["discount_amount"]
line["gross_profit"] = line["net_sales"] - line["quantity"] * line["
    cogs"]
line["discount_pct"] = line["discount_amount"] / line["gross_sales"].
    replace(0, np.nan)
line["discount_bucket"] = pd.cut(
    line["discount_pct"].fillna(0),
    bins=[-0.001, 0.05, 0.10, 0.20, 1.0],
    labels=["<=5%", "5-10%", "10-20%", ">20%"]
)

line["promo_id_str"] = line["promo_id"].astype(str).str.strip()
promotions["promo_id_str"] = promotions["promo_id"].astype(str).str.
    strip()
line = line.merge(
    promotions[["promo_id_str", "promo_type", "promo_channel"]],
    on="promo_id_str",
    how="left"
)
line["promo_type"] = line["promo_type"].fillna("NoPromo")

bucket_perf = line.groupby("discount_bucket", as_index=False).agg(
    net_sales=("net_sales", "sum"),
    gross_profit=("gross_profit", "sum")
)
bucket_perf["margin_rate"] = bucket_perf["gross_profit"] / bucket_perf[
    "net_sales"]

promo_perf = line.groupby("promo_type", as_index=False).agg(
    net_sales=("net_sales", "sum"),
    gross_profit=("gross_profit", "sum")
)
promo_perf["margin_rate"] = promo_perf["gross_profit"] / promo_perf["
    net_sales"]
promo_perf = promo_perf.sort_values("net_sales", ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.barplot(data=bucket_perf, x="discount_bucket", y="margin_rate",
    palette="Blues_r", ax=axes[0])
axes[0].axhline(0, color="black", lw=1)
axes[0].set_title("Margin␣Rate␣by␣Discount␣Bucket")
axes[0].set_xlabel("Discount␣bucket")
axes[0].set_ylabel("Gross␣margin␣rate")

sns.barplot(data=promo_perf, x="promo_type", y="margin_rate", palette="
    Set2", ax=axes[1])
axes[1].axhline(0, color="black", lw=1)
axes[1].set_title("Margin␣Rate␣by␣Promo␣Type")
axes[1].set_xlabel("Promo␣type")
axes[1].set_ylabel("Gross␣margin␣rate")
axes[1].tick_params(axis="x", rotation=20)
```

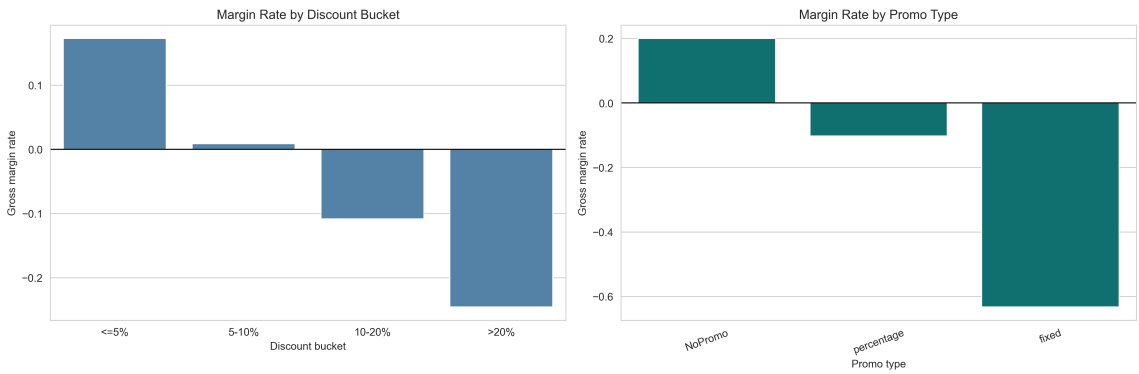

Figure 2: **Promotion profitability diagnostics.** The left panel shows gross margin rate by discount bucket; the right panel shows gross margin rate by promotion type. Margin remains positive at low discount intensity but turns negative beyond the 10% threshold, with fixed-amount promotions producing the deepest losses.

```
plt.tight_layout()
plt.show()
```

2) Data Analysis (Descriptive, Diagnostic, Predictive, Prescriptive) **Descriptive & Diagnostic.** The non-promotion cohort delivers a +19.96% gross margin, while the promotion cohort is negative at -14.46%. By promotion type, *percentage* promos yield -10.25% and *fixed* promos yield -63.28%. By discount intensity, margins remain positive at $\leq 5\%$ (+17.24%), near break-even at 5–10% (+0.87%), and sharply negative above 10%.

Predictive. If the current deep-discount mix is preserved, consolidated margin will continue to deteriorate during peak campaigns.

Prescriptive. Set an explicit discount guardrail at 10%, and allow ¿10% only for overstock SKUs in the top quartile of *days_of_supply*. Policy simulation shows that moving 20% of revenue from the ¿10% band to the 5–10% band can improve gross profit by approximately +96.06 million.

3) Figure caption (NeurIPS format)

1.3 Story 3: High Overstock and Refund Leakage Are Locking Working Capital

1) Visualization Code (Python)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

sns.set_style("whitegrid")

data_dir = Path("./Data") if Path("./Data/inventory.csv").exists() else
    Path("./")
inventory = pd.read_csv(data_dir / "inventory.csv", parse_dates=["
   snapshot_date"])
order_items = pd.read_csv(data_dir / "order_items.csv", low_memory=
   False)
products = pd.read_csv(data_dir / "products.csv")
```

```
returns = pd.read_csv(data_dir / "returns.csv")

inv_latest = inventory[inventory["snapshot_date"] == inventory["
    snapshot_date"].max()].copy()
inv_latest = inv_latest.merge(products[["product_id", "cogs"]], on="
    product_id", how="left")
inv_latest["cogs"] = inv_latest["cogs"].fillna(1.0)
inv_latest["inventory_cost"] = inv_latest["stock_on_hand"] * inv_latest
    ["cogs"]

inv_cat = inv_latest.groupby("category", as_index=False).agg(
    overstock_rate=("overstock_flag", "mean"),
    stockout_rate=("stockout_flag", "mean"),
    avg_dos=("days_of_supply", "mean"),
    inventory_cost=("inventory_cost", "sum")
)

line = order_items.merge(products[["product_id", "category", "cogs"]],
    on="product_id", how="left")
line["discount_amount"] = line["discount_amount"].fillna(0)
line["cogs"] = line["cogs"].fillna(line["unit_price"] * 0.6)
line["net_sales"] = line["quantity"] * line["unit_price"] - line["
    discount_amount"]
line["gross_profit"] = line["net_sales"] - line["quantity"] * line["
    cogs"]

sold_cat = line.groupby("category", as_index=False).agg(
    sold_qty=("quantity", "sum"),
    gross_profit=("gross_profit", "sum")
)
ret_cat = (
    returns
    .merge(products[["product_id", "category"]], on="product_id", how="
        left")
    .groupby("category", as_index=False)
    .agg(return_qty=("return_quantity", "sum"), refund_amount=("
        refund_amount", "sum"))
)

risk = sold_cat.merge(ret_cat, on="category", how="left").merge(inv_cat
    , on="category", how="left")
risk[["return_qty", "refund_amount", "overstock_rate", "stockout_rate",
    "avg_dos", "inventory_cost"]] = (
    risk[["return_qty", "refund_amount", "overstock_rate", "
        stockout_rate", "avg_dos", "inventory_cost"]].fillna(0)
)
risk["return_rate"] = risk["return_qty"] / risk["sold_qty"].replace(0,
    np.nan)
risk["refund_to_gp"] = risk["refund_amount"] / risk["gross_profit"].
    replace(0, np.nan)

fig, ax = plt.subplots(figsize=(11, 6))
sns.scatterplot(
    data=risk,
    x="overstock_rate",
    y="refund_to_gp",
    size="inventory_cost",
    hue="category",
```

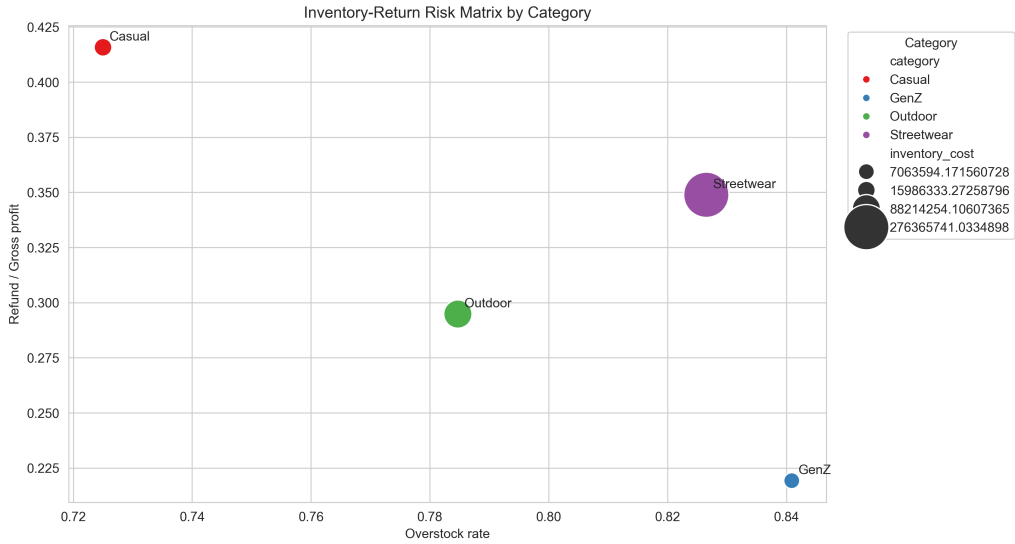

Figure 3: **Inventory-return risk matrix by category.** Bubble size represents inventory cost at the latest snapshot. Categories in the upper-right region combine high overstock rate and high refund-to-gross-profit ratio, signaling simultaneous working-capital lock-in and profitability leakage. The chart identifies priority categories for coordinated pricing, liquidation, and return-mitigation interventions.

```
    sizes=(150, 1200),
    palette="Set1",
    ax=ax
)
for _, r in risk.iterrows():
    ax.annotate(r["category"], (r["overstock_rate"], r["refund_to_gp"])
        , xytext=(5, 5), textcoords="offset␣points")

ax.set_title("Inventory-Return␣Risk␣Matrix␣by␣Category")
ax.set_xlabel("Overstock␣rate")
ax.set_ylabel("Refund␣/␣Gross␣profit")
ax.legend(title="Category", bbox_to_anchor=(1.02, 1), loc="upper␣left")
plt.tight_layout()
plt.show()
```

2) Data Analysis (Descriptive, Diagnostic, Predictive, Prescriptive) **Descriptive & Diagnostic.** Streetwear carries the largest inventory cost (∼276.37 million), with an overstock rate of 82.65% and *refund/gross_profit* of 34.89%. Casual has the highest refund-to-profit leakage (41.57%). GenZ has the highest overstock rate (84.09%), indicating elevated capital lock-in and slower inventory rotation.

Predictive. System-level overstock shows a mild upward trend. Without intervention, working-capital lock-in will persist and constrain reinvestment capacity.

Prescriptive. Prioritize Streetwear and GenZ for margin-aware inventory release programs. Reducing Streetwear refund value by 10%–15% can save approximately 40.67–61.01 million. Lowering Streetwear locked-overstock exposure by 5 percentage points can release roughly 13.82 million in inventory capital.

3) Figure caption (NeurIPS format)

1.4 Executive Synthesis

Across the three stories, the objective is not revenue maximization at any cost, but profit optimization along the joint axis of *Channel x Discount Intensity x Inventory Velocity*. We recommend a weekly control tower with five core KPIs: *orders_per_1k_sessions*, *gross_margin_rate*, *return_rate*, *overstock_rate*, and *refund_to_gp*.